

Kruskal's Algorithm (MST)

DAA Practical 9 — 5th Sem CSE B.Tech

Source Code

```
// Aim: implement Kruskal's algorithm to find the Minimum Spanning Tree (MST)
#include <stdio.h>

#define V 5
#define E 7

struct Edge { int src, dest, weight; };
int parent[V];

int find(int i) {
    while (parent[i] != i) i = parent[i];
    return i;
}

void unionSet(int a, int b) { parent[find(a)] = find(b); }

int main() {
    struct Edge edges[E] = {
        {0, 1, 2}, {0, 3, 6}, {1, 2, 3},
        {1, 3, 8}, {1, 4, 5}, {2, 4, 7}, {3, 4, 9}
    };

    for (int i = 0; i < E - 1; i++)
        for (int j = 0; j < E - i - 1; j++)
            if (edges[j].weight > edges[j + 1].weight) {
                struct Edge t = edges[j]; edges[j] = edges[j + 1]; edges[j + 1] = t;
            }

    for (int i = 0; i < V; i++) parent[i] = i;

    printf("Edges (sorted by weight): src-dest(weight)\n");
    for (int i = 0; i < E; i++)
        printf(" %d-%d(%d)\n", edges[i].src, edges[i].dest, edges[i].weight);

    printf("\nMST edges selected:\n");
    int totalWeight = 0, count = 0;
    for (int i = 0; i < E && count < V - 1; i++) {
        int u = find(edges[i].src), v = find(edges[i].dest);
        if (u != v) {
            printf(" %d -- %d == %d\n", edges[i].src, edges[i].dest, edges[i].weight);
            totalWeight += edges[i].weight;
            unionSet(u, v);
            count++;
        }
    }
    printf("\nTotal MST weight = %d\n", totalWeight);
    return 0;
}
```

Output

```
Edges (sorted by weight): src-dest(weight)
0-1(2)
1-2(3)
1-4(5)
0-3(6)
2-4(7)
1-3(8)
3-4(9)
```

MST edges selected:

```
0 -- 1 == 2
1 -- 2 == 3
1 -- 4 == 5
0 -- 3 == 6
```

Total MST weight = 16